

Steam Games You Can't Miss!!!

Laien Zou, Yara Megahed, Yiwei Levi Pan, Haoxuan Shen

April 2, 2026

Introduction

As game players ourselves, we often find it difficult to quickly discover a game that actually matches what we like. This is especially true for students like us, since we usually do not have the time to go through hundreds of game pages, compare details, and read long reviews. Even though Steam is the world's largest PC gaming platform and provides a huge catalogue with tags and user reviews, we found that its built-in recommendation tools, such as the Interactive Recommender, do not always match our personal preferences. Many of the recommendations feel too broad, too focused on popularity, or not closely related to the specific features we care about. Another issue we noticed is that recommendations are often presented as long, text-heavy lists, which makes them hard to read and compare. Because of this, we started thinking about how recommendations could be made clearer to read and easier to understand. These issues that we faced when trying to find games motivated us to build a system that generates more personalised recommendations and presents them in a clean and visual way, so users can more easily understand the differences between games.

With this in mind, the central question of our project is: **How can we design an interactive system that recommends Steam games to users based on their preferences and presents the results in a clear and interpretable way?** More specifically, the goal of our project is to **implement a graph-based Steam game recommendation system that takes a user's favourite game, preferred genres, and budget range, and generates meaningful recommendations together with interactive visualisations that help explain the results in a clear and understandable manner.** These recommendations are accompanied by useful information, such as review descriptions and prices. In addition, the system highlights how games are related and presents these relationships in a clear and visually intuitive way.

Our system aims not only to recommend games tailored to each user, but also to make the recommendation process more interactive, interpretable, and time-efficient. Our approach first asks users to input a game they are currently enjoying, along with their preferred genres and budget. The output recommendations are not only presented in a traditional list form, but also through a graph, accompanied by useful information such as review descriptions and prices. In addition, the system highlights how games are related and presents these relationships in a clear and visually intuitive way. This allows users to clearly see how recommended games are connected to their favourite game and better understand why those games are suggested.

Datasets

Since Steam is the world’s largest PC gaming platform, and most of us have played games available on Steam, we decided to use data related to Steam games for our project.

Our project uses a combination of real-world Steam game data and a generated user dataset. Since Steam does not provide an official public dataset, we obtained our game data from the public GitHub repository *steam-insights* by NewbieIndieGameDev (NewbieIndieGameDev).

This repository contains multiple CSV files representing different aspects of the Steam platform, including:

- `games.csv`
- `genres.csv`
- `tags.csv`
- `reviews.csv`
- `steamspy_insights.csv`
- `descriptions.csv`
- `promotional.csv`
- `categories.csv`

The original dataset is organized into several separate files, each containing different types of information. For example, `games.csv` contains general game details such as titles and metadata, while `genres.csv` and `tags.csv` provide categorical and descriptive labels for each game. The `reviews.csv` file includes review statistics, and `steamspy_insights.csv` contains additional information such as estimated popularity and playtime. Other files, such as `descriptions.csv`, `promotional.csv`, and `categories.csv`, provide textual descriptions and gameplay-related features.

From these files, we extracted and merged the game attributes needed for our recommendation system into a single unified dataset. The selected attributes include:

- `app_id`: the unique identifier assigned to each game
- `game_name`: the name or title of the game
- `genre`: the genre or genres associated with the game, such as Action, Adventure, Casual, Indie, RPG, Simulation, Sports, Strategy, Free to Play, and Early Access
- `price`: the price of the game
- `platforms`: the platforms supported by the game, such as Windows, Mac, and Linux
- `tags`: descriptive keywords used to characterise the game
- `summary_text`: a short summary or description of the game
- `review_score_description`: a qualitative description of the game’s overall review result, such as “Mostly Positive,” “Very Positive,” or “Overwhelmingly Positive”
- `metacritic_score`: the game’s Metacritic score, if available
- `num_positive_review`: the number of positive reviews the game received
- `num_negative_review`: the number of negative reviews the game received
- `num_recommendation`: the number of users who recommended the game

These attributes were selected because they are the key information our recommendation system relies on. This data about each game allows us to represent games in a way that makes meaningful comparisons possible, which is later used in the filtering and recommendation steps of the program. A more detailed explanation of how these attributes are used is provided in the computational overview section.

Before using the dataset, we performed several data cleaning steps. The original data contained issues such as unreadable text and entries in languages other than English. To ensure consistency, we removed non-English text and filtered out incomplete or invalid records. After cleaning, we also reduced the size of the dataset. The full cleaned dataset would take around 30 minutes to run in our program, which is not efficient. Therefore, we limited the dataset to 4000 rows so that the program can run faster. We included both versions in our submission: the reduced dataset (`filtered_steam_data_4000.csv`) and the larger dataset (`cleaned_steam_data.csv`).

The submitted `data` folder on MarkUs contains several CSV files. The main files used by the final version of our program are `data/filtered_steam_data_4000.csv` and `data/sample_user_data_4000.csv`. These were extracted from the larger cleaned dataset, and `data/cleaned_steam_data.csv` is also included in the submission as the larger cleaned version of the game dataset. In addition, smaller sample files are included for testing and doctest purposes. In particular, the current doctests use `data/medium_game_sample_300.csv` and `data/medium_user_sample_50.csv`. These smaller files are used for doctests because running doctests on the larger datasets would still take a bit too long. Using smaller sample files makes testing quicker and more practical, which also have the same general structure as the full datasets.

In addition to the game dataset, the original dataset did not include detailed user review text linked directly to individual games. Because of this, we used AI to generate a synthetic user dataset. To better reflect a real-world recommendation system, this dataset supports the user–game graph and provides additional contextual information such as user comments. The generated dataset includes the following attributes:

- `user_id`: a unique identifier for each user
- `app_id`: the identifier of the game that the user interacted with
- `comment`: a short user review or feedback generated for the game
- `recommended`: a boolean value indicating whether the user recommends the game or not

1 Computational Overview

The main classes we created and used in our program are:

- **Game**: represents each Steam game and stores its core information
- **Statistics**: stores numerical and review-related data for each game
- **User**: represents users and their review interactions with games
- **_Vertex**: represents a vertex in the graph structure
- **GameUserGraph**: represents the overall graph and handles graph construction and recommendation logic

Together, these classes represent Steam games, user review information, graph vertices, and the overall recommendation graph structure.

1.1 Program Structure

Our program is also organised and divided into several files, each with a different role in the system:

- **main.py**: the entry point of the program; loads data and launches the interface
- **game.py**: defines the **Game** and **Statistics** classes
- **user.py**: defines the **User** class
- **vertex.py**: defines the **_Vertex** class and the similarity logic
- **game_user_graph.py**: builds and manages the graph and recommendation algorithm
- **ui3.py**: handles the Tkinter interface and user interaction flow
- **visualization_graph.py**: generates the interactive graph and positive-review-ratio chart
- **file_reading.py**: handles title cleaning and file reading for game matching

We organized the program in this way to separate different responsibilities within the system. This makes the code easier to understand and maintain, since data representation, graph computation, interface logic, and visualization are handled in different files.

In the following sections, the actual implementation and computation are explained step by step. It begins with how data is represented using classes, followed by how the graph is constructed. After that, the process of calculating similarity between games and generating recommendations is presented. Finally, the user interface and visualization components are discussed.

1.2 Data Representation

As mentioned above, to represent Steam game recommendations, our program uses several classes that store game information, user review data, and the graph structure.

In `game.py`, two main classes are used to represent game data: `Statistics` and `Game`.

The `Statistics` class stores numerical and review-related information for a game, including:

- `review_score_description`: a text description of the game’s overall review result, such as “Very Positive” or “Mostly Positive”
- `metacritic_score`: the game’s Metacritic score, if available
- `num_positive_review`: the number of positive reviews the game received
- `num_negative_review`: the number of negative reviews the game received
- `num_recommendation`: the number of users who recommended the game

The `Game` class stores the main attributes of each game, including:

- `app_id`: the unique identifier of the game in the dataset
- `game_name`: the title of the game
- `game_genre`: the genres associated with the game
- `price`: the price of the game
- `platform`: the list of platforms the game supports, such as Windows, Mac, or Linux
- `tags`: the set of tags describing the game’s features or themes
- `description`: a short summary of the game
- `statistics`: a `Statistics` object containing review-related and numerical information about the game

The `Game` class also includes helper methods such as `get_positive_ratio()` and `is_recommendable()`. These methods are used later in the recommendation process to evaluate review quality and filter out weaker candidates.

In `user.py`, the `User` class represents users in the review dataset. It stores:

- `_user_id`: the unique identifier of the user
- `_comments`: a dictionary storing the comments the user made on different games
- `_recommended`: a dictionary storing whether the user recommended each reviewed game

This user information allows the program to represent user–game relationships and display user comments alongside recommendations in the interface (this will be explained in more detail later).

Together, the `Statistics`, `Game`, and `User` classes form the core data used later for graph construction, similarity computation, filtering, and displaying results.

1.3 Graph Representation

The graph structure is implemented in `vertex.py` and `game_user_graph.py`.

Each `_Vertex` stores:

- `item`: the identifier stored in the vertex, such as a game id or a user id
- `item_type`: indicates whether the vertex represents a game or a user
- `neighbours`: the neighbouring vertices connected to that vertex

The `GameUserGraph` class manages the full graph and stores:

- `_vertices`: a dictionary that stores all vertices in the graph
- `_games`: a dictionary containing all game objects
- `_users`: a dictionary containing all user objects

Our graph contains two main types of vertices:

- game vertices
- user vertices

It also models two kinds of relationships:

- user-game edges, which represent that a user reviewed or interacted with a game
- game-game edges, which represent that two games are considered sufficiently similar according to our similarity function

This graph structure is central to the project because it allows us to model two different but related graph structures within the same system. The first is a user-game graph, where user vertices are connected to game vertices based on reviews or interactions. The second is a game-game graph, where game vertices are connected to other game vertices when they are sufficiently similar according to our similarity function, which will be discussed later. In this way, our project represents both player interaction data and similarity-based relationships between games.

One important implementation issue was that user ids and game ids could overlap numerically. Because both were stored in the same graph structure, this could cause collisions between user vertices and game vertices. To fix this, user ids were stored in a namespaced string format such as `user_123`, which kept the two types of vertices distinct.

1.4 Data Loading and Graph Construction

The dataset is loaded using three major functions in `game_user_graph.py`:

- `load_game_data()`
- `load_user_data()`
- `load_game_user_graph()`

During game-data loading, several helper functions are used to convert raw CSV values into structured `Game` objects. The function `_parse_platform()` converts raw platform fields such as windows, mac, and linux into a cleaned list of supported platforms. The function `_parse_tags()` extracts and cleans game tags from the dataset so that they can later be used in similarity calculations. The function `_to_int_or_none()` converts numerical fields into integers when possible, and handles missing values by returning `None`. Finally, `_create_statistics()` combines the review-related columns into a `Statistics` object, which is then stored inside each `Game` object. Together, these helper functions transform raw dataset rows into consistent objects that can be used directly by the graph and recommendation system.

Then, `load_game_user_graph()` builds the full graph by:

- adding all game vertices
- adding all user vertices
- connecting users to games they reviewed
- adding game-game edges between sufficiently similar games

1.5 Similarity Computation

The similarity score between two games is computed in the `_Vertex` class using the `similarity_score()` method. The system mainly uses a Jaccard-style similarity measure to compare the tag sets of the two games (“Jaccard Similarity”). In particular, the tag score is calculated by taking the number of shared tags and dividing it by the total number of distinct tags across both games. In addition to this, the system also considers platform and genre similarity. The platform score is set to 1 if the two games share at least one platform, and 0 otherwise. Similarly, the genre score is 1 if the two games share at least one genre, and 0 otherwise.

These components are then combined into a final similarity score using a weighted formula: 0.8 for the tag score, 0.1 for the platform score, and 0.1 for the genre score. This means that tag similarity has the largest impact, while platform and genre provide smaller supporting contributions.

Using the `get_similarity()` function, two games are considered similar if their similarity score is at least 0.35. This threshold allows the system to connect games based on multiple shared features, instead of relying on just a single attribute.

1.6 User Interface and Output

The recommendation process begins in the Tkinter interface, which is the main graphical user interface of our program. The interface is organized into several pages, and the user moves through them using navigation buttons such as **Start Survey**, **Continue**, **Back**, and **Back to Home**. The home page introduces the Steam game finder and allows the user to begin the survey.

On the first question page, the user is asked: “What is your favourite Steam game, one that you are currently playing?” The interface also tells the user to enter only one game. If the user tries to continue without entering anything, an error message appears saying “Please enter a game!” and the program does not allow them to proceed. After the user enters text, the program cleans the input by converting it to lowercase and removing punctuation so that it can be matched with game titles in the dataset. In addition, the matching logic chooses the longest valid match when multiple possible matches exist. We used this approach because some games share very similar words in their titles, even though they are different games. For example, titles such as Counter-Strike and Counter-Strike 2 share the same initial words. If the system simply selected the first valid match it found, it could incorrectly match the user’s input to the shorter title instead of the correct one.

On the second question page, the user is asked: “What genres do you usually enjoy? Pick as many as you like.” This page uses multiple checkboxes, and the user can select more than one genre without having to type everything manually. We chose this design because the genres in our dataset already appear as a fixed set of categories, including Action, Adventure, Casual, Indie, Massively Multiplayer, Racing, RPG, Simulation, Sports, Strategy, Free to Play, and Early Access. Turning these into multiple-choice options makes the interface easier for the user to complete and also makes it easier for the implementation to process genre input. If the user tries to continue without selecting any genre, the interface shows the error message “Please choose at least one genre!”

On the third question page, the user is asked: “What is your budget?” The budget range is implemented using two sliders, one for the minimum price and one for the maximum price. We chose this design because it allows the user to select a price range quickly and easily without typing numbers manually. The minimum slider is initialized to 0 and the maximum slider is initialized to 250. If the user sets the minimum price greater than or equal to the maximum price, the interface shows the error message “minimum price cannot be greater than or equal to maximum price!” and prevents the user from continuing. This ensures that the selected price range is valid before the recommendation system generates results.

After each valid input is submitted, the user’s answer is also printed in the console. This allows the user to make sure that their response is being correctly stored and recorded by the system. In addition, the interface includes a **Back to Home** button on all of the main pages. If the user returns to the home page in this way, all previous inputs and stored answers are cleared automatically, including the entered game, selected genres, budget range, current recommendation list, and selected center game. This allows the user to restart the recommendation process smoothly without needing to quit and rerun the program.

After the user provides valid inputs, the program moves to the results page. This page displays up to ten recommended games in a scrollable text box. For each game, the page shows information such as the title, genres, price, and review description. This page also includes buttons that allow the user to move to the visualization page, return to the previous page, or return to the home page.

1.7 Visualization

In addition to the recommendation list, the visualization page provides the next stage of output. The system includes two main visual components:

- an interactive recommendation graph
- a positive-review-ratio bar chart

The recommendation graph highlights the centre game in red and the recommended games in blue. Games with higher similarity are placed closer to the center, helping users more clearly understand the relationships between the center game and the recommended games. When the user moves the mouse over a node, a box appears showing detailed information about that game. This includes the game name, price, genres, supported platforms, and review description. For recommended games, the similarity score relative to the center game is also shown. If available, a user comment is included as well.

The positive-review-ratio bar chart compares the review quality of the center game and the displayed recommended games. More specifically, the chart shows the positive review ratio for each displayed game, which allows the user to compare how well each game is received based on player reviews for the games we recommended. In the final version of the chart, the game with the highest positive review ratio is highlighted in red, and the remaining games are shown in blue. This helps the user quickly identify which displayed game has the strongest review quality.

The system also includes a comments page that displays all the comments from previous users who played the selected and recommended games. These comments are shown in a scrollable text area so that longer comments can still be read clearly within the interface. The comments page also includes navigation buttons, allowing the user to go back to the visualization page or return directly to the home page.

1.8 Identifying the Centre Game

To determine the center game, the system first checks whether the user’s entered game exists in the dataset. This decision is handled in `ui3.py`. If the entered game is found, then the user’s favourite game becomes the center game directly. The recommendation system then starts from that game vertex and uses its neighbouring game vertices in the game-game graph as candidate recommendations.

If the entered game is not found in the dataset, the system cannot use it directly as the starting point. In this case, the program selects a fallback center game based on the user’s other preferences. In `ui3.py`, this is done by first collecting candidate games that match the user’s selected genres and budget range, and that also satisfy the recommendation filter (positive review ratio greater than 0.70 and total number of reviews greater than 500). The candidate games are then ranked in a fixed way so that the system always chooses the same “strongest” fallback game instead of choosing one randomly. More specifically, the fallback game is selected by prioritizing higher positive review ratio, then higher total number of reviews, and finally a smaller `app_id` if needed. This selected game then becomes the center game for the recommendation process.

Once the center game has been determined, the main recommendation logic is carried out in `game_user_graph.py`. In this process, `_collect_scored_games()` gathers neighbouring game vertices and their similarity scores, while `_matches_price_range()` checks whether each candidate fits within the user’s selected budget range. In the final version of our system, recommendations are generated by starting from the center game vertex and examining only its neighbouring game vertices in the graph. The candidate games are then filtered by selected genres and recommendation quality, sorted by similarity score, and returned as up to 10 recommended games. This is important because it means the recommendation system uses the game-game graph structure that we constructed, rather than comparing the selected game with every game in the dataset.

To improve the quality of the final results, we apply an additional recommendation constraint using review-based information. More specifically, a game is only considered recommendable if its positive review ratio is greater than 0.70 and its total number of reviews is greater than 500. We added this filter for two reasons. First, it improves recommendation quality by reducing the chance that poorly reviewed or unreliable games appear in the

final recommendation list. Second, because the dataset is quite large, this filtering step helps narrow the set of candidate games before the final ranking stage. In other words, it allows the system to focus on games that have both stronger review performance and enough review data to be considered more trustworthy.

1.9 Use of Libraries

Our project uses several Python libraries to support its computation, interface, and visualization.

Tkinter The `tkinter` library is used to build the graphical user interface of the program. More specifically, we use widgets such as `Frame`, `Label`, `Button`, `Entry`, `Checkbutton`, `Scale`, `Text`, and `Scrollbar` to create the survey pages, collect user input, display recommendation results, and organize navigation between pages. In addition, the data types `IntVar` and `BooleanVar` are used to store the state of sliders and checkboxes in the interface. These features make `tkinter` suitable for building the multi-page interactive survey and result display used in our program.

NetworkX The `networkx` library is used in `visualization_graph.py` to build a local graph structure for visualization. Although the main recommendation system uses our own graph classes, `networkx` is useful for storing the center game and recommended game connections in a form that is convenient for graph visualization. In particular, we use the `Graph` data type from `networkx` to represent the local recommendation graph before it is displayed.

Plotly The `plotly` library is used to generate the interactive browser-based visualizations of the project. More specifically, we use `plotly.graph_objects` to create both the recommendation graph and the positive-review-ratio chart. Functions and data types such as `go.Figure`, `go.Scatter`, and `go.Bar` are used to display nodes, edges, and bar charts interactively. This is especially important because it allows the program to present results visually in a clearer way than a plain text list.

Pillow The `Pillow` library is used to load and display the image on the home page of the Tkinter interface. In particular, `Image.open()` is used to load the image file, and `ImageTk.PhotoImage()` is used to convert it into a format that can be displayed inside a Tkinter widget. This allows the interface to include visual design elements in addition to text and buttons.

Together, these libraries make it possible for our project to go beyond simple text-based output and provide a recommendation system that is interactive, visual, and easier for users to understand.

2 Instructions for Obtaining Datasets and Running Your Program

Our project is written entirely in Python and was developed to run with Python 3.13.5. Before running the program, the TA should first install the required external Python libraries listed in the `requirements.txt` file. The required external libraries are:

- `python-ta`
- `plotly`
- `networkx`
- `Pillow`

No additional programming languages or non-Python software are required to run our project.

2.1 Dataset Setup

All datasets used by our program are included in a single zip file called `data.zip` prepared for submission. This zip file contains all datasets needed by the project, including the filtered and cleaned datasets used by the final version of the recommendation system, as well as the generated user dataset.

Our TA can obtain the dataset zip file through the following method:

- the zip file is uploaded to MarkUs as part of our final project submission

After downloading the zip file, the TA should extract its contents into the same folder as the project files. Our final project expects all files, including the datasets, to be stored in the project root folder.

2.2 Running the Program

Make sure that all the files(including image) are in the same folder. Once the required libraries are installed and the dataset files have been extracted, the TA should run the file `main.py` or run the file in the console. This launches the Tkinter graphical user interface of our Steam recommendation system.

2.3 Expected Output

The TA should expect to see:

- a home page introducing the Steam Game Finder
- a sequence of survey-style pages asking for:
 - a favourite Steam game
 - preferred genres
 - a budget range
- a results page displaying the recommended games in a scrollable text box

For each recommended game, the results page displays:

- game title
- genres
- price
- review description

From the results page, the TA can:

- open the interactive recommendation graph
- open the positive-review-ratio chart
- move to the comments page

The recommendation graph opens in the browser and displays:

- the center game in red
- the recommended games in blue
- distance from the center based on similarity

Games with higher similarity are placed closer to the center game.

The positive-review-ratio chart also opens in the browser. It compares the positive review ratio of the center game and the displayed recommended games, and the game with the highest positive review ratio is highlighted in red.

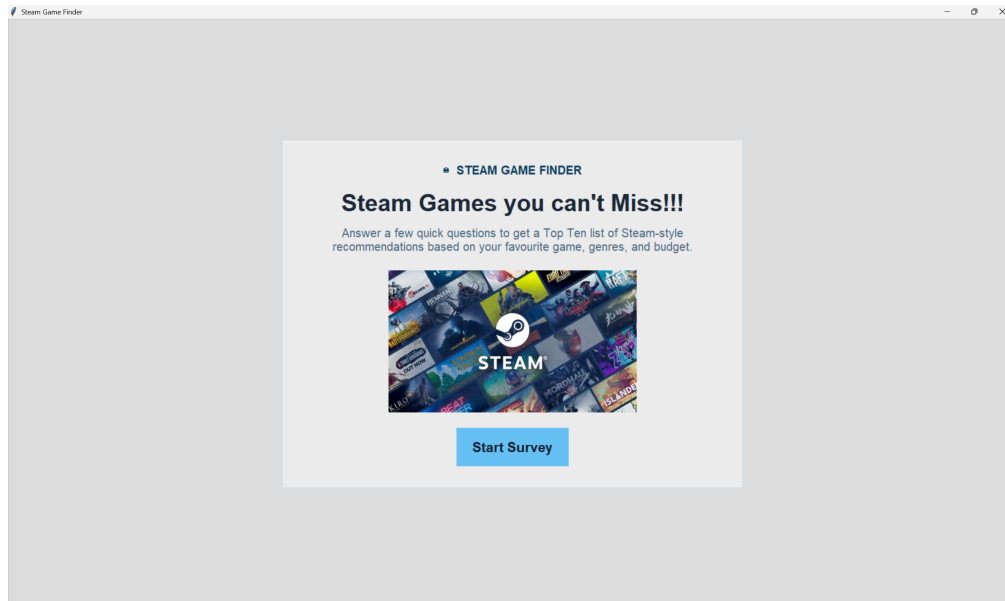
The comments page remains inside the Tkinter interface and displays comments from previous users who played the recommended games.

2.4 Example Walkthrough

Example 1: Entered Game Found in the Dataset

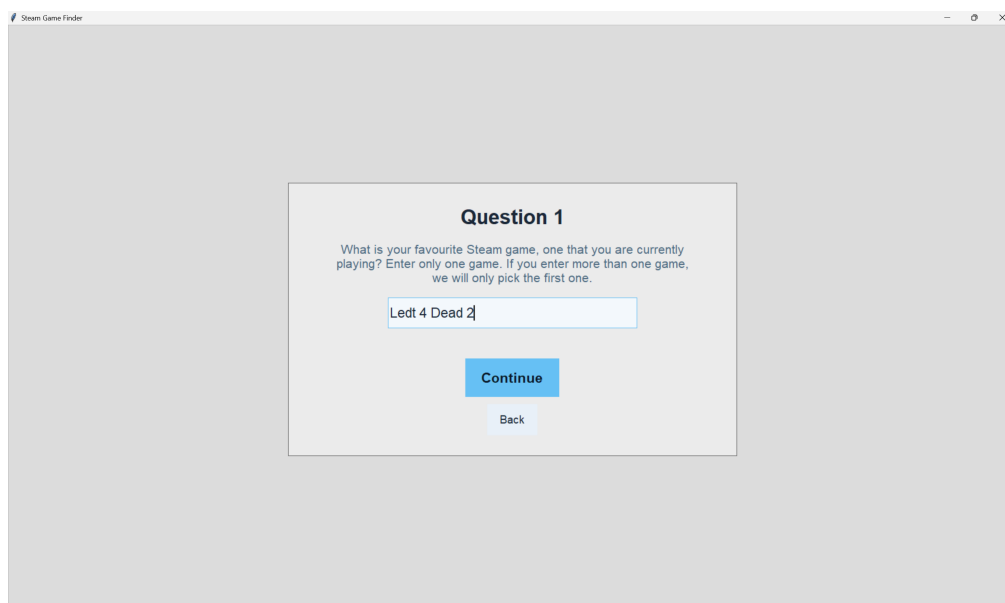
Step 1. Home page

The user opens the program and begins on the home page of the Steam Game Finder. From here, the user presses **Start Survey** to begin the recommendation process.



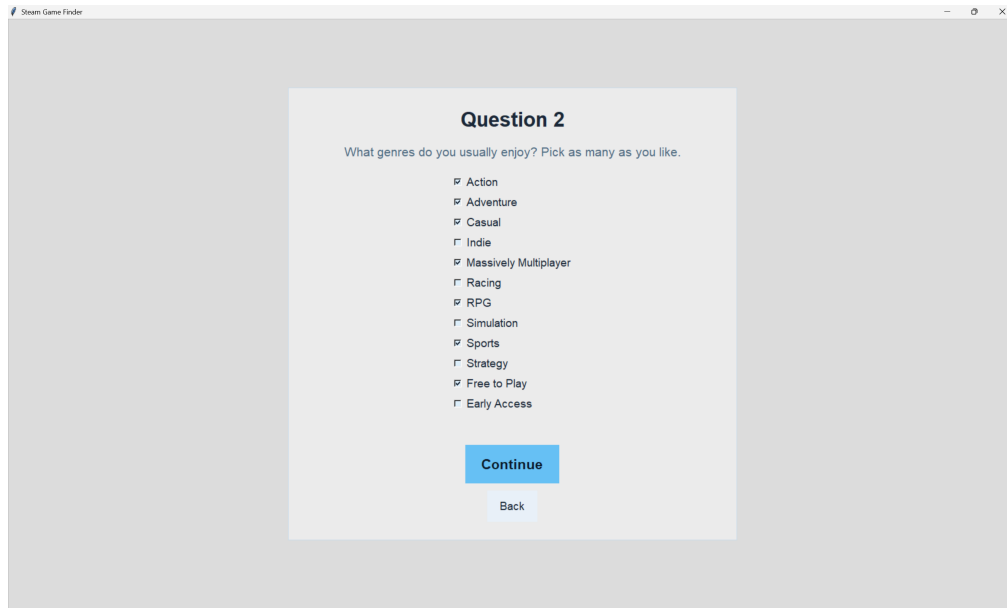
Step 2. Question 1: Favourite Steam game

The user enters **Left 4 Dead 2** as their favourite Steam game. Since this game exists in the dataset, it will later be used as the center game of the recommendation process.



Step 3. Question 2: Genre selection

The user selects their preferred genres. In this example, all genres are selected.



Question 2

What genres do you usually enjoy? Pick as many as you like.

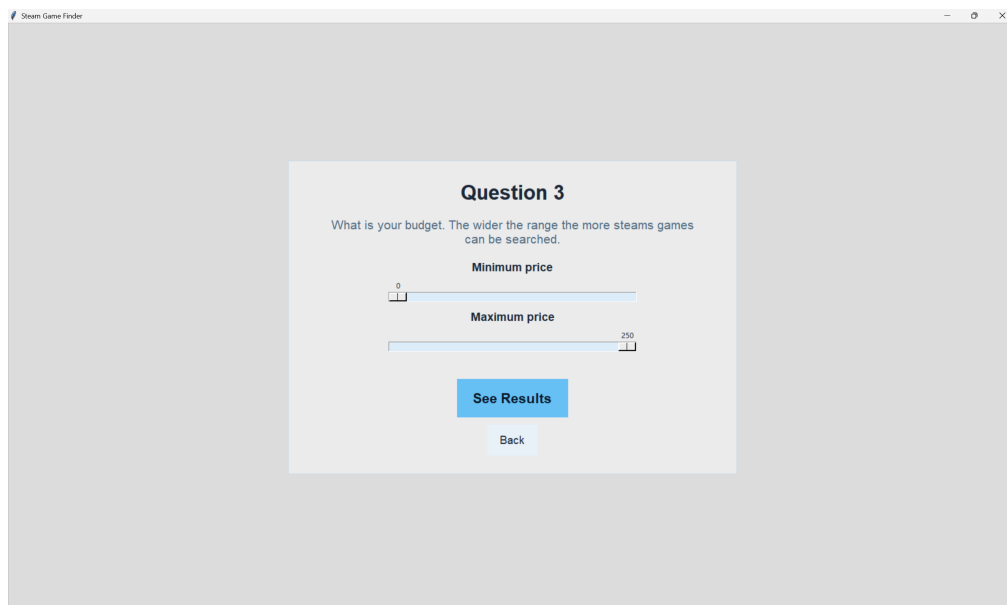
- ☒ Action
- ☒ Adventure
- ☒ Casual
- ☒ Indie
- ☒ Massively Multiplayer
- ☒ Racing
- ☒ RPG
- ☒ Simulation
- ☒ Sports
- ☒ Strategy
- ☒ Free to Play
- ☒ Early Access

Continue

Back

Step 4. Question 3: Budget range

The user leaves the budget range unchanged, using the default values.



Question 3

What is your budget. The wider the range the more steams games can be searched.

Minimum price

0

Maximum price

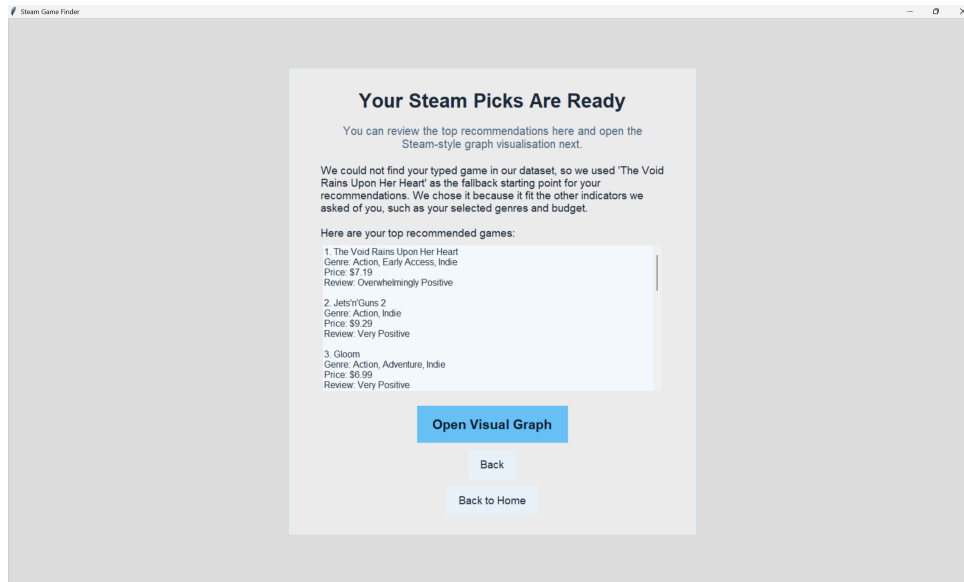
250

See Results

Back

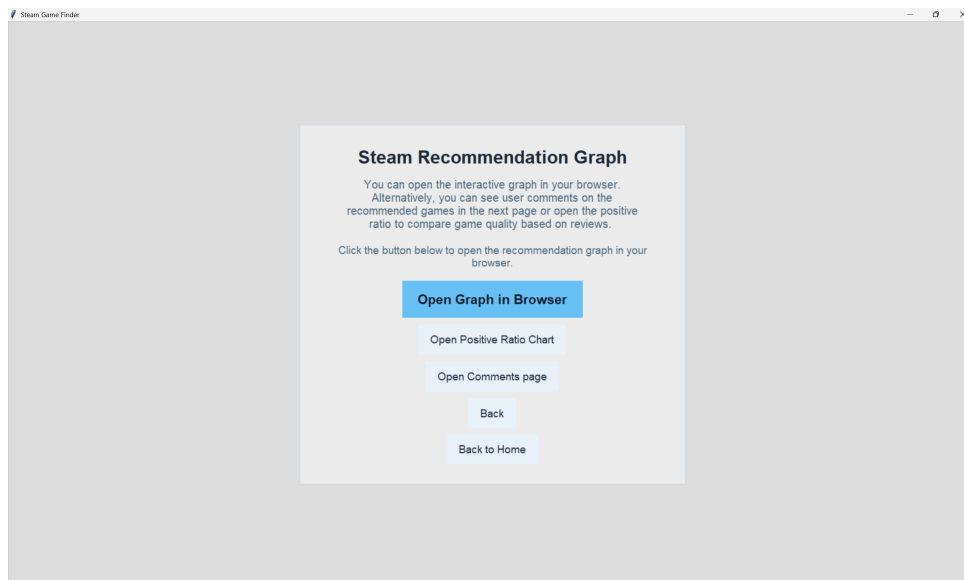
Step 5. Results page

The system displays the recommendation results in a scrollable text box. Each recommended game includes information such as title, genres, price, and review description.



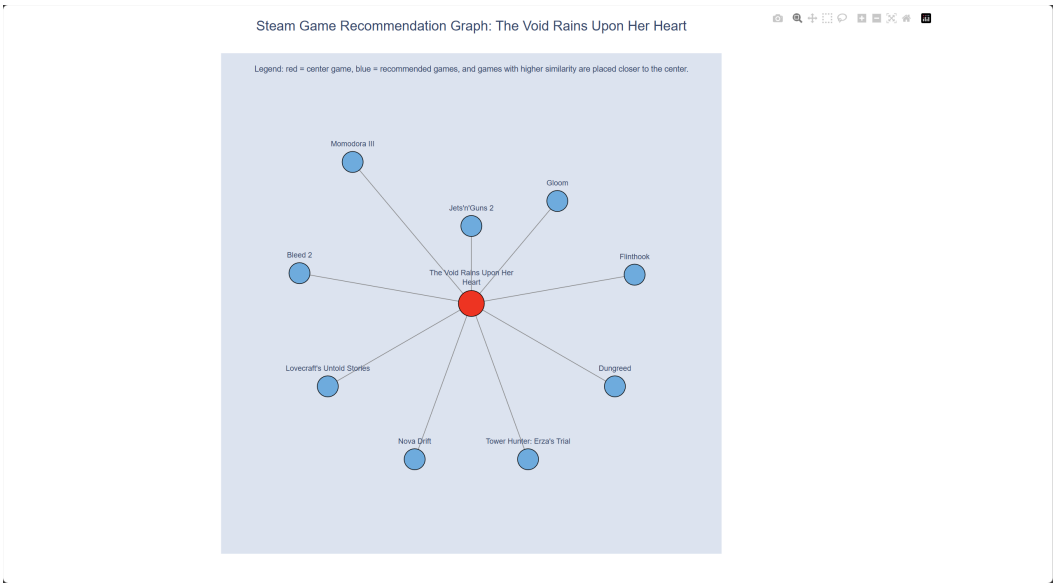
Step 6. Visualization page

The user moves to the visualization page, where they can choose to open the recommendation graph in the browser, open the positive-review-ratio chart, or move to the comments page.



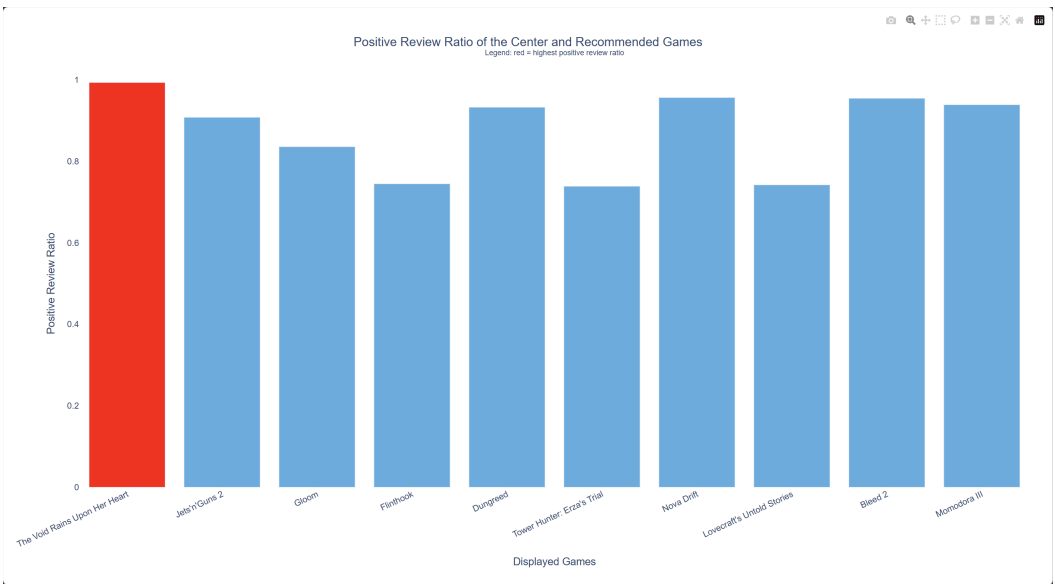
Step 7. Recommendation graph

The user opens the visual recommendation graph. The selected center game is shown in red, the recommended games are shown in blue, and games with higher similarity are placed closer to the center.



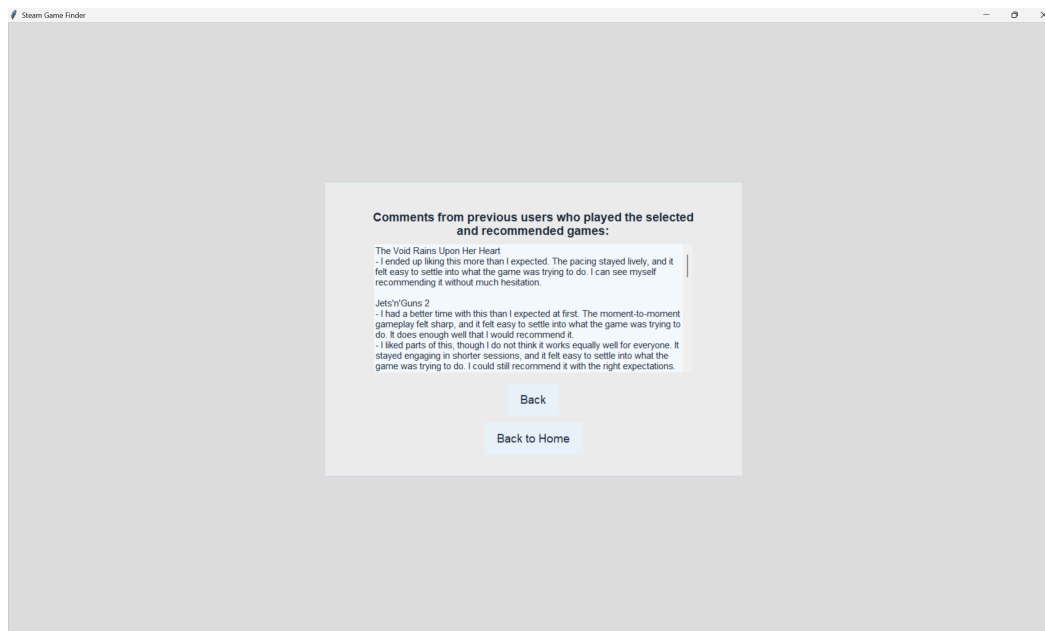
Step 8. Positive-review-ratio chart

The user opens the positive-review-ratio chart to compare the review quality of the center game and the displayed recommended games.



Step 9. Comments page

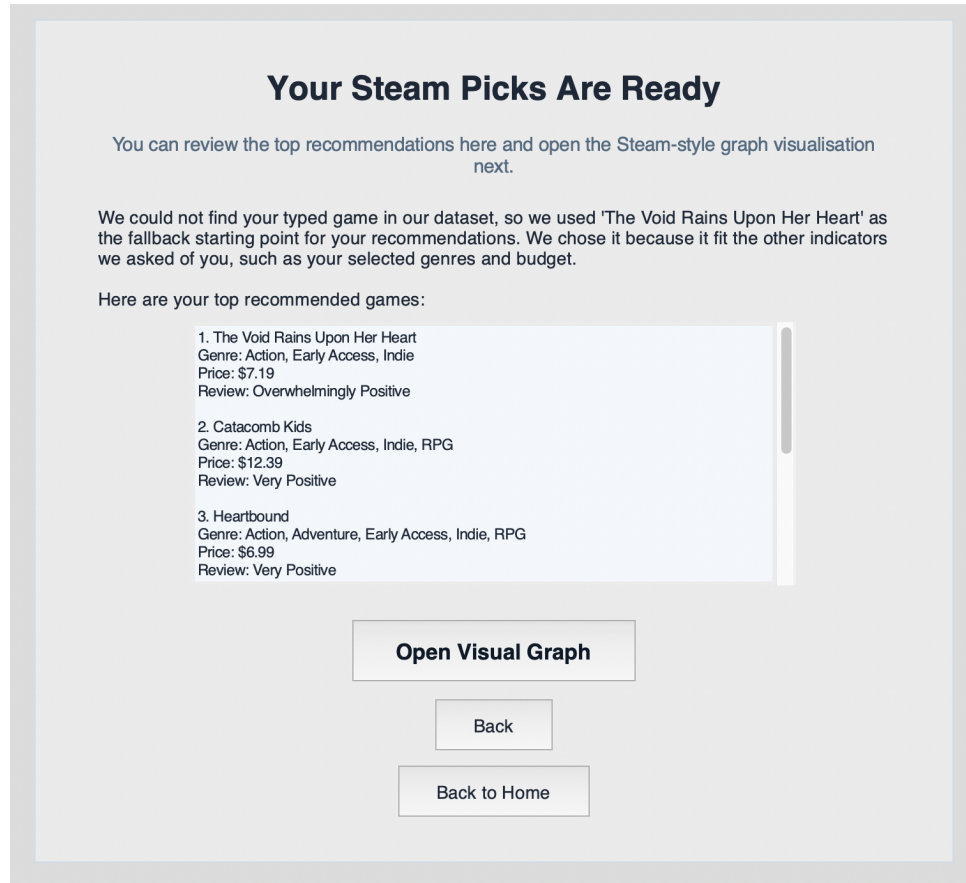
The user opens the comments page to view comments from previous users who played the selected and recommended games.



Example 2: Entered Game Not Found in the Dataset

Step 1. Question 1: Entering a game not in the dataset

The user enters a game name that does not exist in the dataset, such as **aaa**.



Step 2. Genre and budget selection

The user selects any genres and any budget range. These inputs will be used to determine the fallback starting game.

Step 3. Results page with fallback starting game

Since the entered game is not in the dataset, the system chooses a fallback starting game that matches the selected filters. The results page explains this and then displays the recommendations generated from that fallback center game.

Your Steam Picks Are Ready

You can review the top recommendations here and open the Steam-style graph visualisation next.

We could not find your typed game in our dataset, so we used 'The Void Rains Upon Her Heart' as the fallback starting point for your recommendations. We chose it because it fit the other indicators we asked of you, such as your selected genres and budget.

Here are your top recommended games:

1. The Void Rains Upon Her Heart
Genre: Action, Early Access, Indie
Price: \$7.19
Review: Overwhelmingly Positive
2. Catacomb Kids
Genre: Action, Early Access, Indie, RPG
Price: \$12.39
Review: Very Positive
3. Heartbound
Genre: Action, Adventure, Early Access, Indie, RPG
Price: \$6.99
Review: Very Positive

Open Visual Graph

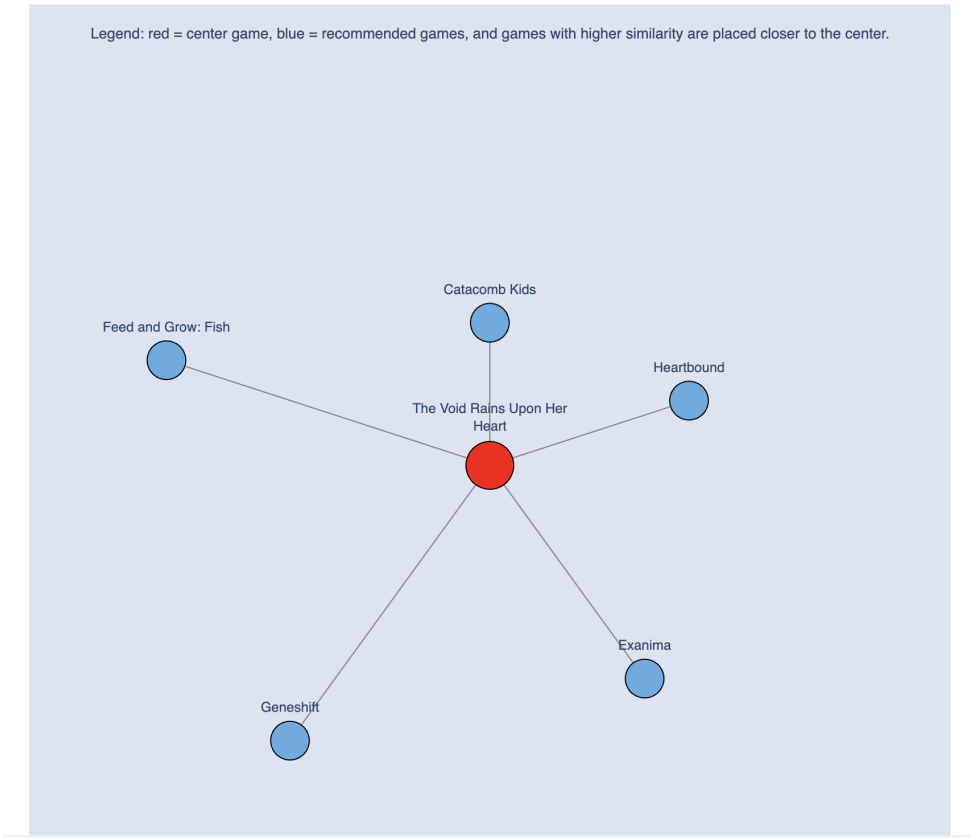
Back

Back to Home

Step 4. Visualization page for fallback results

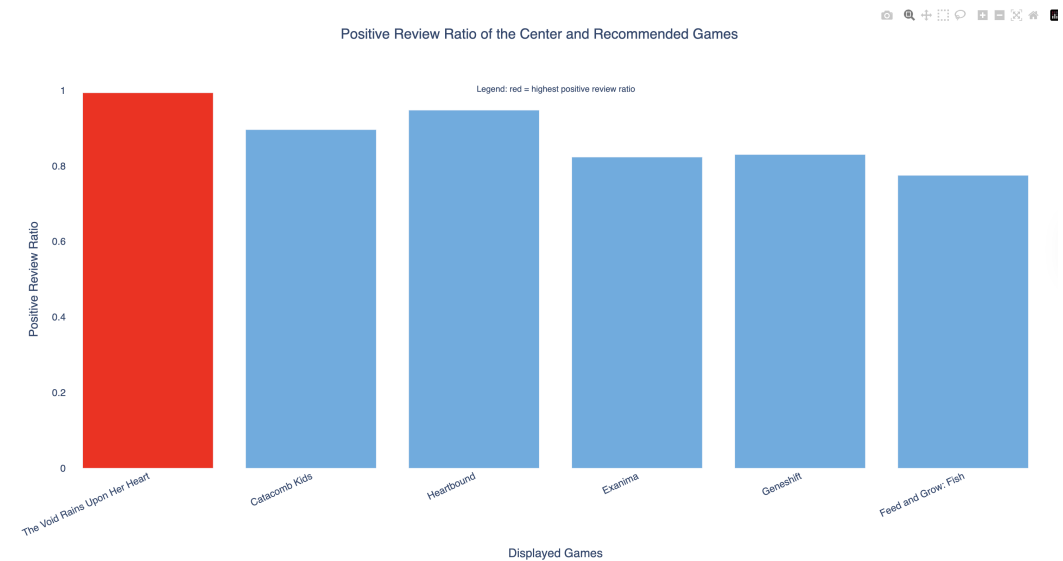
The user can then move to the visualization page for the fallback-based recommendations.

Steam Game Recommendation Graph: The Void Rains Upon Her Heart



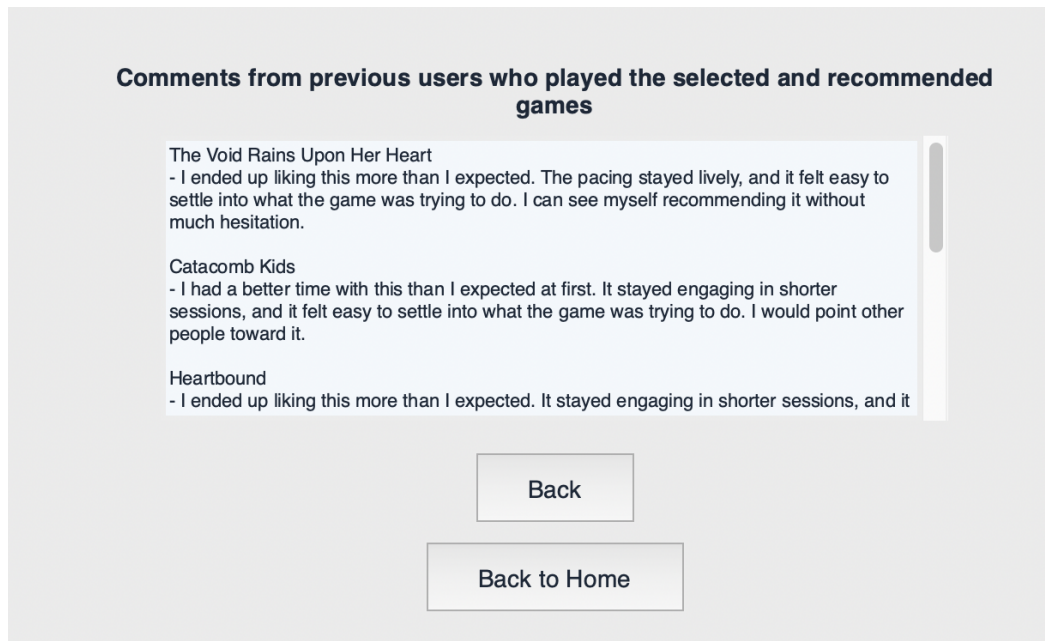
Step 5. Recommendation graph for fallback results

The user opens the graph for the fallback-based recommendations.



Step 6. Comments page for fallback results

The user opens the comments page for the fallback-based recommendations.



Changes Between Proposal and Final Submission

Although the central goal of our project remained the same, several important parts of the project changed between the proposal stage and the final version.

Change 1: Recommendation System Refinement and Filtering Order

First, while the main recommendation idea stayed the same, we refined the similarity and filtering system in order to produce better recommendations. In our proposal, we planned to recommend games mainly based on similarity between games. In the final version, we refined the system based on TA feedback by adding additional filtering conditions instead of relying only on similarity scores. In particular, we first apply genre filtering, then use similarity to compare games, and finally apply a recommendation filter based on positive review ratio and total number of reviews.

One important issue we encountered was the order in which filtering was applied during recommendation. In an earlier version of the system, the program first selected the most similar games and only afterwards filtered those results based on the user's selected genres. This created an unexpected problem: sometimes a wider budget range produced no recommendations, while a narrower budget range produced valid results. The reason was that with a wider budget, the initially selected top similar games could all fail the later genre filter, which caused the final list to become empty. However, with a narrower budget, some of those higher-ranked games were removed earlier by the price filter, allowing other games that matched the selected genres to appear in the final results. To fix this issue, we changed the recommendation process so that genre filtering is applied before selecting the final top recommendations. This made the output more stable and more consistent with the user's expectations.

We also added another filtering step based on positive review ratio and total number of reviews. This was an important improvement because it reduces the number of low-quality or low-information games in the recommendation results. As a result, the final recommendation system is not only based on similarity, but also considers the overall quality and reliability of each game's review data. As suggested by the TA, one possible enhancement was to implement different scoring systems and compare them. Instead of combining everything into one similarity score, we chose to separate the process into stages: first filtering by genre, then ranking by similarity, and finally filtering by positive review ratio. We believe this approach is more practical because genre and review conditions act more like yes-or-no checks rather than continuous similarity values.

Summary of Changes

1. The recommendation system is no longer based only on similarity scores.
2. Additional filtering conditions were added, including genre filtering, positive review ratio, and total number of reviews.
3. The order of filtering was changed so that genre filtering is applied before similarity ranking.
4. This change fixed the issue where wider budget ranges sometimes produced no recommendations.
5. The final recommendation system is more stable and produces more consistent results.

Change 2: Graph Structure Instead of Tree Structure

Second, in our original proposal, we considered using both graph and tree structures, including the idea of organizing games in a genre tree. After further discussion, we decided not to include a tree structure in the final version. A tree works best when items belong to a clear hierarchy, but game recommendations are not naturally structured in this way. A single game can belong to several genres at the same time and can also be related to other games through tags, platforms, and review characteristics. Because of this, a tree would be too restrictive for our project. In contrast, a graph-based approach allows one game to connect to many other games through different shared characteristics at the same time. This made the graph a much better fit for our recommendation system. For the same reason, we also changed the interface slightly so that users can select multiple genres instead of being forced to choose only one category.

Summary of Change 2

1. Games can belong to multiple genres at the same time.
2. A tree structure is too restrictive for this type of relationship.
3. The interface was updated to allow users to select multiple genres.

Change 3: Visualization and Recommendation Presentation

Another major change was in how we present recommendations to the user. In the proposal, we considered showing the “path” between games to explain why certain games were recommended. The goal was to make the system more transparent by showing how the recommendation was generated. However, after testing this idea, we found that displaying too many neighbouring games and connections made the visualization cluttered and difficult to understand. Instead, we implemented interactive visualizations that were clearer and easier to interpret. The final system includes a recommendation graph and a positive-review-ratio chart. In the graph, games with higher similarity are placed closer to the center game, which helps users visually understand how the recommendations are related. This still achieves our original goal of making the system more interpretable, but in a much clearer and more effective way. The positive-review-ratio chart provides additional support, which allows users to compare which games are reviewed more positively.

Summary of Change 3

1. The path visualization was too cluttered and difficult to understand.
2. We replaced it with an interactive recommendation graph. and added a positive-review-ratio chart to compare game quality.

Change 4: User Interaction Flow

As we have touched on this earlier, we also revised the user interaction flow during implementation. Initially, we considered relying more on general preferences such as genres and budget. However, since our recommendation system works best when it starts from a specific game, we redesigned the interface so that it begins with the user's favourite game, together with their preferred genres and budget. This allows the system to generate recommendations around a game the user already likes, rather than relying only on broad preference categories. To support this design, we also added a fallback system for cases where the user's entered game does not exist in the dataset. In that case, the system selects a suitable starting game based on the user's other preferences and then generates recommendations from that point. This change became an important design decision in the final version because it made the recommendation process more meaningful. Since the system begins from a game the user already likes, it is more likely to generate results that match the user's actual interests.

Summary of Change 4 The user interaction flow was updated as follows:

1. The system now starts from the user's favourite game.
2. The system also considers preferred genres and budget.
3. A fallback system was added if the entered game is not in the dataset.
4. Recommendations are generated around a game the user already likes.

Discussion

Overall, we believe our final program achieved the main goal of the project: building an interactive Steam game recommendation system that suggests games based on a user's favourite game, preferred genres, and budget. Rather than relying only on simple filtering, our system uses graph-based relationships and similarity scores to find games that are more closely connected to the user's interests. We also wanted the results to feel clearer and easier to understand, so instead of showing only a text list, we included an interactive recommendation graph, a positive-review-ratio chart, and a comments page. These features help users understand the recommendations quickly and clearly.

The results of our computational approach do help answer our project question. At the beginning of the project, we wanted to find a way to design an interactive system that recommends Steam games based on a user's preferences and presents the results in a clear and understandable way. In the final version, we believe our system does this well by combining a graph-based recommendation model with user-selected filters such as genre and budget. If the user's favourite game exists in the dataset, the system uses that game as the starting point and recommends nearby similar games. If the entered game is not found in the dataset, the system chooses a fallback starting game based on the user's other preferences and generates recommendations from there. This makes the recommendation process feel more logical and increases the chance of finding games that better match the user's interests, because the system always begins from a game the user already likes.

One important strength of our final project is that the recommendation process is easier to understand compared to many recommendation systems users would typically see online, which often only show a list of results. Instead of only displaying game names, the visualisation allows users to clearly see the centre game and how each recommended game is connected to it. The distance between the games shows how similar each recommended game is. Games that are closer to the centre are more similar, while games that are further away are less similar. In addition, the positive-review-ratio chart gives users another way to compare the recommended games by showing how well each game is received by players. The comments page also adds valuable information by showing feedback from previous users, which helps users better understand what each game is like. Together, these features make the results easier to understand and make the recommendation process more transparent to the user, which helps explain why certain games are recommended.

Project Limitations

1. Dataset Limitations

However, our project also has several important limitations. One of the biggest limitations comes from the dataset itself. In our implementation, we only use a filtered dataset of 4000 games, which is a very small subset compared to the full range of games available on Steam. This is because using the full dataset made the program take too long to run, as the graph construction and similarity computations were very time-consuming. Because of this, we reduced the dataset size so that the program can run in a reasonable amount of time. However, this also means the recommendation system is limited to a much smaller pool of titles, which makes the recommendation output more limited. Even though the newest games in the dataset are from 2024, we noticed that many of the games were released before 2019, meaning that newer and more popular titles are often missing. This becomes a problem when users are looking for more recent or trending games, as the system may not be able to recommend them at all.

2. Performance and Graph Construction Limitations

Another big limitation we ran into was how long it takes to build the graph. When we tried using the full dataset, the program took too long to run, which made it impractical to use. Because of this, we had to reduce the dataset size in the final version. While this makes the system much faster and easier to work with, it also means we are limiting the range of games the system can recommend. This was a tradeoff we had to make between performance and recommendation quality.

3. Recommendation Model Limitations

Our computational selection of recommended games is still fairly simple and does not fully reflect how real-world systems change over time. Since we are using a fixed dataset, the system cannot account for things like discounts, regional pricing, or free-to-play changes that happen often on Steam. We also only use a small set of inputs from the user, namely their favourite game, preferred genres, and budget. In reality, people's preferences are much more complex. Because of this, the system may miss important factors such as preferred difficulty, art style, game length, and other features that influence whether a user would actually enjoy a game.

Future Improvements

1. Graph Construction Optimization

There are several directions we could take to improve this project in the future. One thing we would like to improve is the graph construction. Right now, building the graph takes a long time when the dataset is large, so we had to limit the number of games. If we could optimize this process, we would be able to use a much larger dataset, which would make the system more practical to use in real life.

2. Using More Up-to-Date Data

Another improvement would be to use more up-to-date data. Currently, our dataset is fixed and does not always include newer or more popular games. In the future, we would want to include more recent Steam data, or even explore connecting the system to a real-time data source such as an API. This would allow the recommendation system to stay updated and make the results much more useful for users in real-world settings.

3. Improving the Recommendation Logic

We also think the recommendation logic itself could be improved further. At the moment, the system mainly relies on tags, genres, platforms, and review-based information. While this works reasonably well, it does not capture everything that users care about. In the future, we could include more features such as playtime information, difficulty level, and more detailed review text. Taking more factors into account would likely improve the quality of the recommendations.

4. Improving the Budget System

Another area we would like to improve is the budget system. Since the dataset uses prices in pounds, the current filter is based on that currency, which can be confusing for users in different regions. A better version of the system could include currency conversion so that users can understand and use the budget range more easily.

In conclusion, although our system has clear limitations, we believe it still achieved the main goal of the project. We were able to build an interactive Steam game recommendation system that is based on graphs and presents its results through visual outputs, making the recommendations more understandable than a simple text list. At the same time, working on this project helped us realize that the dataset behind the system is just as important as the implementation of the recommendation algorithm itself. Even when the algorithm works as intended, the quality of the final recommendations is still heavily dependent on the quality of the dataset. This was something we did not fully realize at the beginning of the project, but it became much clearer as we used the interface from a user's perspective. Because our dataset is limited in size and contains many older games, the system cannot always recommend the most relevant or up-to-date titles, even if the recommendation logic itself is reasonable. In that sense, one of the most important things we learned from this project is that good recommendation results do not depend only on algorithm design, but also on the effort put into finding, cleaning, and organising data in a way that makes it ready to use and useful in real-world situations.

References by Category

GitHub Sources

NewbieIndieGameDev. *steam-insights*. GitHub,
<https://github.com/NewbieIndieGameDev/steam-insights>. Accessed 2 Apr. 2026.

Academic Sources

Jaccard Similarity. Elsevier, 2024,
<https://www.sciencedirect.com/topics/computer-science/jaccard-similarity>.

LaTeX Tutorials

Overleaf. *Bibliography Management in LaTeX*. May 2021,
https://www.overleaf.com/learn/latex/Bibliography_management_in_LaTeX.

Reminder

The `project_report.zip` is the file that contains `project_report.tex` and the images used in `Latex`. Make sure to upload all the images and tex file to Latex if TA wants to compile the pdf by tex file.